

Quant Research Lab — Research Report

1 Quant Research Lab — Research Report

This report is the single narrative for the lab: alternative-data stock screening, probability and learner diagnostics, market simulation, technical indicators, rule-based and machine-learned trading strategies, reinforcement learning, and agent-based market simulation.

How to read the sections. Where results evolved, each block is labeled:

- **First implementation / first test / initial results** — original design and metrics
- **After improvements / updated results** — re-runs on the current package, data protocol, and evaluation discipline

Where useful: *X was tested and Y was received before improvements were made.*

Figures live under `docs/img/report/` and `docs/img/`. Executable detail is in `notebooks/`. A LaTeX twin of this document is `RESEARCH_REPORT.tex`.

1.1 1. Preface

Research question. Can alternative data and machine learning find — and exploit — an edge in equities, once costs and out-of-sample evaluation are taken seriously?

The pipeline grew from a fundamentals + Glassdoor screen into a cost-aware backtesting lab with from-scratch learners and RL. Early trading experiments used shorter crisis-era windows and fixed ± 1000 -share position caps. The improved protocol standardizes on longer train/test splits, unlevered sizing, walk-forward checks, and an installable `quantlab` package with unit tests.

1.2 2. Stock investment and Glassdoor alternative data

1.2.1 2.1 Goal

Develop a database and model for stock investment decisions by compiling public-company information (fundamentals plus employee-sourced outlook) and turning it into auditable screens and statistical tests.

1.2.2 2.2 Hypothesis

Does Glassdoor *Positive Business Outlook* correlate with long-run company ROI?

First test. On the 2020 research snapshots (Morningstar fundamentals joined to Glassdoor outlook/rating), OLS and Spearman analysis rejected the null of no association: $p = 1e-10$ with $n = 548$, but $r^2 = 0.07$. Outlook behaves like a *screening factor*, not a standalone trading signal.

Caveats recorded at the time:

- Associational look-back returns (not a true forward prediction study)
- Survivorship bias in the scraped universe
- No claim of post-2020 predictive validity
- Correlation is not causation

After improvements. The same hypothesis utilities live in `quantlab.stats` (Cochran sample sizing, Spearman matrices with p-values, OLS with explicit reject/fail-to-reject). Loaders in `quantlab.data.fundamentals` clean messy CSV tokens, thousands separators, yes/no/ESG flags, and trend labels. Notebook `01_hypothesis_and_research_design.ipynb` reproduces the verdict; provenance for the original scraper is under `docs/archive/`.

1.2.3 2.3 Data pipeline

First implementation. Web scraping assembled monthly fundamentals + outlook tables (examples committed as `data/2020_09.csv`, `2020_11.csv`, `2020_12.csv`).

After improvements. A single tested loading path versions those snapshots; prices come from Yahoo Finance with a deterministic local cache (`quantlab.data.prices`). Notebook `02_data_pipeline.ipynb`.

1.2.4 2.4 Screening rules

First test. Eight auditable rules compressed roughly **1,170** names to about a **15-name** list (e.g. AAPL, MSFT, NVDA):

1. Net income per employee $\geq \$10,000$
2. Net profit margin $> 2.57\%$
3. 3-, 5-, and 10-year trailing ROI all $> 10.1\%$
4. Price shows an upward trend
5. Glassdoor positive business outlook $\geq 60\%$
6. Glassdoor overall rating ≥ 3.5
7. PEG ratio between 0 and 1
8. Beats S&P 500 over 1 year and 5 years

After improvements. Rules are a typed engine in `quantlab.screening` with unit tests; notebook `03_screening_rules.ipynb`.

1.3 3. Martingale (American roulette)

1.3.1 3.1 Abstract / setup

A martingale is a stochastic process where the expectation of the next value equals the present value. Here the scenario is gambling: a gambler's winnings form a martingale if games are fair. The practical idea is that because you cannot lose every time, you increase the stake after losses in anticipation of a future win that recovers prior losses.

Setup. Always bet on black on an American roulette wheel (0 and 00): 38 pockets, 18 black (P(win) = 18/38 47.7%). Double the stake after each loss; quit at +\$80 or after 1000 sequential bets.

Single-bet expected value (unit stake):

$$\mathbb{E} = \frac{18}{38}(+1) + \frac{20}{38}(-1) \approx -\$0.05$$

so the house edge is about five cents per dollar bet.

1.3.2 3.2 Experiment 1 — unlimited bankroll (first test)

Estimated probability of winning \$80 within 1000 bets.

All 1000 simulations achieved +\$80 estimated probability **100%**. Repeated doubling with an unlimited bank account always recovers to the prior high and then freezes at the +\$80 target.

Figure note (first test): the first 300 bets of 10 simulations all reached \$80 by roughly 200 sequential bets.

Estimated expected value after 1000 bets.

Because every path hits +\$80 and freezes, the mean terminal value across simulations is **\$80**. Equivalently: the path is a sequence of geometric progressions ending in a win that reverses losses, then quitting at the target.

Do mean $\pm$ SD lines reach a maximum then stabilize / converge?

Yes. Once equity freezes at \$80, standard deviation $\rightarrow 0$, so mean, mean+SD, and mean−SD stabilize and **converge**. SD rises then falls as more paths lock in.

Median $\pm$ SD plots behave similarly under the unlimited-bankroll freeze.

1.3.3 3.3 Experiment 2 — \$256 bankroll cap (first test)

Estimated probability of winning \$80 within 1000 bets.

666 of 1000 simulations succeeded **66.6%**. The capital cap blocks recovery after a deep losing streak. With single-spin win rate 47.7%, more trials would be expected to push this estimate somewhat lower.

Estimated expected value after 1000 bets.

Paths converge near +\$80 (~66.6%) or −\$256 (~33.3%):

$$0.666 \times 80 + 0.334 \times (-256) \approx -\$34.53$$

If hit rate falls with more trials, EV becomes more negative.

Do mean $\pm$ SD lines converge?

No. First-test plateaus were roughly: mean+SD \$126.24, mean -\$32.22, mean-SD -\$190.69, with SD \$158.46. Because a large mass remains at -\$256, SD never approaches 0 and the bands do not converge.

1.3.4 3.4 After improvements

Clean API in `quantlab.sim.martingale` with seeded RNG. Re-run (200 sims, seed 42) via `scripts/generate_report_assets.py`:

Setting	Hit rate	Expected terminal
Unlimited	1.00	80.0
Cap \$256	0.63	-44.3

Conclusions match the first test (sure win with infinite capital; negative EV with a finite bankroll). Differences vs 1000-sim first-test figures are Monte Carlo noise. See `martingale_paths.png` and notebook `12_martingale_and_gridworld.ipynb`.

1.4 4. Portfolio optimization

1.4.1 4.1 Method

Maximize the Sharpe ratio of a long-only, fully invested portfolio with SciPy SLSQP. Daily returns of the weighted book are used; the objective is typically minimizing the negative Sharpe.

1.4.2 4.2 First implementation

Parameters

Parameter	Value
Start	2008-06-01
End	2009-06-01
Symbols	JPM, GLD, X, IBM

Optimizer / portfolio results (first test)

Metric	Value
Current function value	-0.026653247
Iterations	7
Function evaluations	42
Gradient evaluations	7
Allocations	[1, 0, 0, 0] (100% JPM)
Sum of allocations	1.00
Sharpe ratio	-1.0939
Volatility (daily)	reported alongside objective

Metric	Value
Average daily return	0.0018367
Cumulative return	−0.1148

Normalized optimal portfolio vs S&P 500 was plotted for the crisis window. The takeaway: in-sample “optimal” weights in a crash do not imply a good absolute outcome — here the Sharpe-maximizing book was essentially all JPM and still lost money.

1.4.3 4.3 After improvements

Notebook `05_portfolio_optimization.ipynb` uses train **2016–2019** / OOS **2020–2023**, often on the screened universe, benchmarked to SPY and equal weight (`quantlab.portfolio`). Max-Sharpe can beat SPY out-of-sample, but the in-sample edge decays — the same lesson as the first test, under a healthier evaluation protocol.

1.5 5. Assess learners

1.5.1 5.1 Abstract and introduction

Compare from-scratch learners that accept tabular (non-time-series) inputs and return continuous predictions: **decision tree**, **random tree**, **bag**, **InsaneLearner**, and **linear regression**. Primary metric: RMSE. Later experiments also use MAE and wall-clock time.

Initial hypotheses (first test):

1. Increasing leaf size decreases overfitting but increases error.
2. Decision trees take longer than random trees but are more accurate, because they choose splits by correlation rather than at random.

1.5.2 5.2 Methods

Data: `Istanbul.csv` (also accepted: any similar numeric matrix). Shuffle, then **60%** train / **40%** test. Leaf sizes swept **1–100** where relevant. Bagging defaults to **20** bags and can wrap any base learner.

Split rule: decision trees pick the feature with highest absolute correlation with the target and split at the median; random trees pick a random feature.

1.5.3 5.3 Experiment 1 — leaf size vs overfitting (decision tree)

Overfitting means fitting training data so closely that the model fails on new data. Gauge it by **in-sample vs out-of-sample RMSE**.

Smaller leaf size → deeper trees → more overfit. First-test Figure 1 showed very low in-sample RMSE and high out-of-sample RMSE at small leaves. The smallest in/out RMSE *gap* occurred near **leaf size 21**; at that point and smaller, overfit was visible. Larger leaves constrain depth and improve generalization.

1.5.4 5.4 Experiment 2 — bagging

Bagging draws bootstrap subsets, trains a learner per bag, and averages predictions to reduce variance. First-test Figure 2 (20 bags) showed lower RMSE at both low and high leaf sizes than bare DT (e.g. leaves 10, 21, 81), and a smaller in/out gap at very small leaves — bagging **reduces** overfit impact but does **not** eliminate it (an overfit region remained for leaf 81). Varying bag count would not remove that region entirely.

1.5.5 5.5 Experiment 3 — decision tree vs random tree

Metrics: MAE and time to fit + query train and test (X).

Expectation: RT less accurate, often less overfit-prone in practice as a weak learner, and faster. First-test results: DT MAE generally better; RT up to $\sim 4\times$ **faster** at small leaves, with the gap narrowing as leaf size grows. Bagged RTs inherit the speed advantage.

1.5.6 5.6 Non-experiment component — InsaneLearner

InsaneLearner = **20** bag learners, each a bag of **20** linear regressors (400 linear models), implemented compactly. Linear regression uses least squares with an intercept column.

1.5.7 5.7 First-test summary

Inverse relationship between leaf size and overfitting; RMSE gap diagnoses overfit; bagging helps; DT vs RT is an accuracy/speed tradeoff; **no single model wins everywhere**.

1.5.8 5.8 After improvements

Same experiments in `quantlab.experiments` + notebook `11_assess_learners.ipynb`. Figures: `learners_leaf_rmse.png`, `learners_dt_vs_rt.png`.

Synthetic **best-for-*** datasets (`quantlab.experiments.best4`): linear targets favor OLS (near-zero RMSE vs large tree error); axis-aligned thresholds favor trees — concrete proof that “best learner” depends on data geometry.

1.6 6. Market simulator

1.6.1 6.1 First implementation

Order-file simulator: dated BUY/SELL rows with symbol and share count; mark portfolio value daily. Cost model: flat **commission** per fill and fractional **impact** (buys pay more, sells receive less).

Typical early strategy settings: commission **\$9.95**, impact **0.005**, positions in $\{-1000, 0, +1000\}$, start cash **\$100,000**.

1.6.2 6.2 After improvements

- `quantlab.backtest.run_backtest` — single-asset trade series

- `quantlab.backtest.run_orders` — multi-symbol order logs
- Metrics: Sharpe, max drawdown, cumulative/avg daily return, volatility

Default research costs in later notebooks: **\$1 commission + 0.1% impact**, with **unlevered** share caps via `affordable_shares`.

1.7 7. Indicators and theoretically optimal strategy

Charts historically used **adjusted close normalized to 1.0** at the start of the window. Implementations live in `quantlab.indicators`.

1.7.1 7.1 Simple moving average (SMA)

SMA is the arithmetic mean of the last (n) prices:

```
prices.rolling(n).mean()
```

It smooths noise and lags price. Traders watch short SMA crossing above long SMA as a possible uptrend start (and the reverse for downtrend).

First test: (n {20, 50}) on JPM; multiple 20/50 crosses over ~2 years; 20 above 50 interpreted as upward trend.

1.7.2 7.2 Bollinger Bands

Upper/lower bands at $SMA \pm (\text{width} \times \text{rolling standard deviation})$. First test: (n = 20), width = 2.

- **Squeeze** (bands close): low recent vol; often precedes a vol expansion, but does not by itself give direction or timing.
- Price near lower band: often read as oversold; near upper: overbought. Bands alone are not sufficient trading signals (Bollinger's own guidance).

%B / band value locates price relative to the bands. First-test plots scaled the band value ($\div 10$) for readability; late-2009 showed bands tightening (falling SD).

1.7.3 7.3 Momentum

Leading rate-of-change vs price (n) periods ago (first test: (n = 20)):

$$\text{momentum}_t = \frac{P_t}{P_{t-n}} - 1$$

Positive bullish momentum. Crossing up through zero after being below does not prove a downtrend is over — only that it is slowing. First-test figure: momentum spiked after the ~2009-04 trend reversal.

1.7.4 7.4 Exponential moving average (EMA)

EMA weights recent prices more heavily:

1. Seed with SMA
2. Multiplier ($= 2/(N+1)$)
3. $EMA_t = P_t \cdot m + EMA_{t-1} \cdot (1 - m)$

```
prices.ewm(...).mean()
```

First test used 20- and 50-day EMA; 20 crossing above 50 as a buy cue. EMA crosses can lag differently than SMA crosses on the same chart.

1.7.5 7.5 Volatility

Dispersion of returns; higher vol wider range of outcomes (riskier in that sense). First indicator study used **7-day** rolling std of daily returns ($\times 2$ for display); strategy work often used **20-day**. High vol early-2009 coincided with the large downtrend. Risk-averse traders may refuse to trade when vol is extreme; others combine high vol with trend direction as a signal.

1.7.6 7.6 Signal research (after improvements)

Notebook 04_signal_research.ipynb: information coefficients vs forward returns are consistent for some mean-reversion features but tiny ($|IC| < 0.1$), so **costs dominate**.

1.7.7 7.7 Theoretically optimal strategy (TOS)

Assumptions. Perfect foresight of tomorrow's close; no bankroll limit; trades at adjusted close; at most one trade/day; positions in $\{-1000, 0, +1000\}$; **zero** commission and impact.

Construction (first test). Mark each day $+1/-1$ by whether the next close is higher/lower. When that sign flips, trade to flip between long and short (share deltas up to ± 2000 while net holdings stay in $\{-1000, 0, +1000\}$).

Window: JPM, 2008-01-01 $\rightarrow$ 2009-12-31, \$100,000 start.

Benchmark: buy 1000 shares day one and hold (normalized to 1.0).

First-test performance

Metric	TOS	Benchmark
Sharpe ratio	13.3228	0.1569
Cumulative return	5.7861	0.0123
Stdev of daily returns	0.0045	0.0170
Average daily return	0.0038	0.0002
Final value	\$678,610	\$101,230

Benchmark stayed relatively flat; TOS trended strongly upward — an upper bound, not a tradable policy.

After improvements. `theoretically_optimal_trades` in `quantlab.strategies`; notebook 06 uses it as a ceiling. Illustration: `tos_ceiling.png`.

1.8 8. Q-learning

1.8.1 8.1 First test — grid worlds

Tabular Q-learning (optional **Dyna** replay) on discrete mazes (`data/gridworlds/`). Actions: N/E/S/W. Rewards: step penalty, large penalty for pits, positive reward at goal. The agent learns a path from start to goal.

1.8.2 8.2 After improvements — trading application

Same `QLearner` drives `TradingEnvironment`: quantile-binned indicators + position in the state; reward = position $\times$ next return $-$ impact. Notebook `08_rl_trading_agent.ipynb`: learns a conservative policy but is too coarse to beat buy-and-hold on MSFT OOS. Notebook `12_martingale_and_gridworld.ipynb` keeps maze training available.

1.9 9. Strategy evaluation

Initial hypothesis (first test): Manual Strategy beats Benchmark **in-sample**; Strategy Learner beats Manual.

1.9.1 9.1 Indicators used for strategies

Same family as §7 (SMA 20/50, Bollinger ($n=20$), width 2, momentum 20, volatility 20-day $\times 2$ for display/signals). Prices forward- then back-filled and normalized to 1.0 at start for the first-test charts.

1.9.2 9.2 Manual strategy — creation and signals (first implementation)

Windows: in-sample 2008-01-01 $\rightarrow$ 2009-12-31; out-of-sample 2010-01-01 $\rightarrow$ 2011-12-31. Symbol JPM. Holdings constrained to $\{-1000, 0, +1000\}$.

Actions sum component votes: sum > 0 buy, sum < 0 sell, 0 flat. Multiple indicators hedge single-indicator failure and allow trading vs pure hold.

+1 / buy when any of:

1. 20-day SMA crosses **above** 50-day SMA (today above, yesterday below).
2. Price crosses **above** the lower Bollinger band (today above, yesterday below).
3. Momentum crosses **above** zero (today > 0 , yesterday < 0 , with the prior-day stability check used in the first implementation).

-1 / sell when any of:

1. 20-day SMA crosses **below** 50-day SMA.
2. Price crosses **below** the upper Bollinger band.
3. Momentum crosses **below** zero (symmetric to the buy rule).

Example: votes (+1, −1, 0) sum to 0 — no trade.

1.9.3 9.3 Manual vs benchmark — first-test performance

In-sample: Manual beat Benchmark (as designed). Out-of-sample: Manual **worse** than Benchmark — thresholds tuned on in-sample did not transfer. “Past performance doesn’t guarantee future performance.”

Metric	Manual (In)	Bench (In)	Manual (Out)	Bench (Out)
Sharpe	0.188	0.153	−1.4569	−0.2636
Cumulative return	0.034	0.01	−0.3847	−0.0853
Stdev daily return	0.0151	0.017	0.00997	0.0085
Avg daily return	0.000179	0.00016	−0.0009	−0.00014
Number of trades	41	2	40	2
Final value	\$103,340.80	\$100,819.25	\$61,517.55	\$91,273.10

Extra trades explain in-sample outperformance vs a static hold; the same activity hurt out-of-sample.

1.9.4 9.4 Strategy learner (first implementation)

Features: same indicators as Manual (incl. 20/50 SMA ratio, Bollinger %B, momentum, volatility).

Target: 5-day percent return, discretized to {−1, 0, +1} with thresholds that include impact. Missing features filled with 0.

Model: BagLearner of Random Trees — **20 bags, leaf size 6** (larger leaf to limit degradation). Train 2008–2009; test 2010–2011.

1.9.5 9.5 Experiment 1 — Manual vs Learner vs Benchmark (first test)

Commission **9.5**, impact **0.005**, \$100k start, ±1000 shares, fixed RNG seed.

Hypothesis: Learner beats Manual — **supported** in- and out-of-sample vs Manual. Out-of-sample, **Benchmark still beat the Learner**; all three finished below start in that OOS window. Without a seed, RT bags would not reproduce exactly; other OOS windows might change rankings.

In-sample table (first test)

Metric	Strategy Learner	Manual	Benchmark
Sharpe	3.6	0.188	0.153
Cumulative return	1.816	0.034	0.01
Stdev daily return	0.009	0.0151	0.017
Avg daily return	0.0021	0.000179	0.00016

Metric	Strategy Learner	Manual	Benchmark
Number of trades	117	41	2
Final value	\$281,056.30	\$103,340.80	\$100,819.25

In-sample, the Learner had the best return and Sharpe with the **lowest** daily vol while trading the **most** — classic in-sample success that did not fully carry OOS against buy-and-hold.

1.9.6 9.6 Experiment 2 — impact sensitivity (first test)

Commission **0**; impacts **0, 0.002, 0.004, 0.006**. Hypothesis: larger impact → lower final value (impact is a cost). Generally true for cum return / avg daily return / final value; not perfectly monotone at tiny impact steps because of random trees. Higher impact tended to reduce Sharpe. At impact 0.02, trades became minimal; high enough impact can yield **no** trades.

In-sample impact table (first test)

Impact	0	0.002	0.004	0.006
Sharpe	3.676	3.857	3.901	3.535
Cumulative return	1.92	2.023	2.043	1.776
Stdev daily	0.00938	0.00921	0.00916	0.0093
Avg daily return	0.00217	0.00223	0.00225	0.00207
Number of trades	93	109	103	112
Final value	\$292,030	\$302,036	\$303,881	\$276,948

1.9.7 9.7 After improvements

Protocol: **MSFT**, train **2016–2019**, OOS **2020–2023**, \$100k, unlevered shares, **\$1 + 0.1%** impact. Manual rules use vote thresholds in `ManualRuleStrategy`; ML uses bagged random trees in `MLTradingStrategy`. Walk-forward yearly checks show the ML edge is **not** year-consistent.

Strategy	Cumulative return	Sharpe	Max drawdown	Final value
ML learner (bagged random trees)	+185.4%	1.02	−27.0%	\$285,412
Buy & hold	+143.0%	0.85	−37.2%	\$242,714
Manual rules	+33.1%	0.38	−45.4%	\$133,046
Q-learner	+6.1%	0.17	−31.4%	\$106,103

Impact grids remain in notebook 06. Limitations: single-symbol strategy evaluation, simple cost model, 2020 fundamentals vintage.

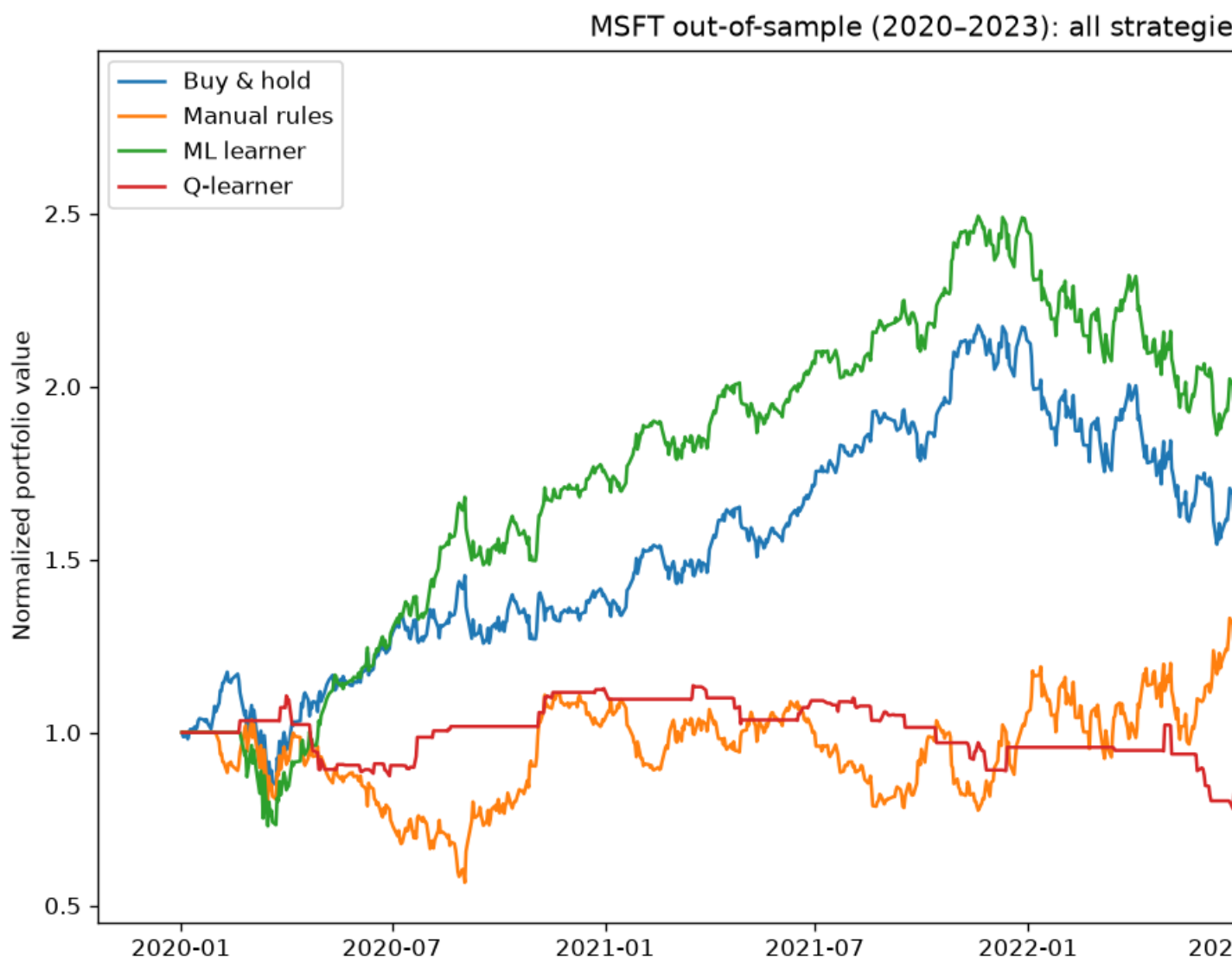


Figure 1: Final scorecard

1.10 10. ABIDES market simulation

1.10.1 10.1 First implementation — agent design

Custom agent in the Agent-Based Interactive Discrete Event Simulation environment (EMA mid-price momentum + inventory-aware quoting):

1. On wake: cancel resting orders; fetch spread / book.
2. Compare fast vs slow mid-price EMAs → buy / sell / flat momentum status.
3. Size and price using depth-weighted mid estimates and recent bid–ask spread volatility; inventory skew raises bids when flat/cash-heavy and lowers asks when long.
4. Place an order only when momentum and inventory logic agree; pause after fills to avoid over-leverage / order spam.
5. Flatten within ~5 minutes of the close.

1.10.2 10.2 After improvements

Kernel vendored under `third_party/abides/` (see `PROVENANCE.md` and upstream `LICENSE`). Strategy logic also as testable pure functions in `quantlab.abides_agent`. Notebook `10_abides_market_simulation.ipynb` and `quantlab.abides_runner` (`pip install -e ".[abides]"`).

1.11 11. Unified conclusions

1. **Glassdoor outlook** associates with historical ROI weakly (screening factor, not standalone alpha).
2. **Finite-bankroll martingales** have negative expected value despite frequent small wins.
3. **Learner choice** depends on leaf size, bagging, and data geometry.
4. **TOS** shows how large a frictionless edge looks; real strategies must clear costs.
5. **Hand rules** overfit short in-sample windows; **bagged trees** can beat buy-and-hold on a modern OOS window but fail walk-forward consistency.
6. **Tabular RL** learns sensible low-risk behavior, not a benchmark-beating policy under the current state design.
7. **ABIDES** remains the path for microstructure-style agent experiments.

Next steps (notebook 09): cross-sectional ML on the full screen, deeper RL, fresher alternative data, richer execution realism.

1.12 References

1. Hayes, A. (2020, August 28). Bollinger Band®. <https://www.investopedia.com/terms/b/bollingerbands.asp>
2. Hayes, A. (2020, September 10). Exponential Moving Average (EMA). <https://www.investopedia.com/terms>
3. Hayes, A. (2020, September 22). Simple Moving Average (SMA) Definition. <https://www.investopedia.com/t>
4. Staff, I. (2020, August 28). Momentum Indicates Stock Price Strength. <https://www.investopedia.com/article>
5. Woods, G. (2019, September 26). Trading with the Bollinger Band Squeeze. <https://www.tradingsetupsreview.com/bollinger-band-squeeze/>
6. Byrd, D., Hybinette, M., & Balch, T. *ABIDES: Towards High-Fidelity Market Simulation for AI Research*. arXiv:1904.12066.